**MAHATMA GANDHI UNIVERSITY, KOTTAYAM**
**MGU-UGP (HONOURS) REGULAR EXAMINATION March 2026**
**Fourth SEMESTER**
**Discipline Specific Core Course (DSC) - MG4DSCECT200 - PYTHON PROGRAMMING (2024 ADMISSION ONWARDS)**

**SCHEME OF EVALUATION -QP CODE 25901224**

**Duration: 1 hour 30 minutes** **Maximum Marks: 50**

***Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I)*and Appreciation(Ap)**

***Students should attempt at least one question from each course outcome to enhance their overall* outcome attainability.**

**Part A**
**MCQ**
**Answer all questions**
**Each question carries 1 marks**

1. a) **bool**
2. d) **elif**
3. d) **import package**
4. c) **//**
5. a) **To terminate the program execution**
6. c) **BaseException**
7. c) **Both command line and IDE**
8. c) **break**
9. c) **Any single lowercase letter from a to z**
10. b) **plot()**

**Part B**
**Short Answer Question**
**Answer any 4 questions**
**Each question carries 5 marks**

**11. Explain dictionaries in Python and discuss any two common dictionary methods.**

A dictionary in Python is a collection of key–value pairs. It is unordered, mutable, and indexed by keys.
Syntax: student = {"name": "Anu", "age": 20, "course": "BCA"}
Example: print(student["name"]) # Output: Anu

**Syntax:**

```
student = {"name": "Anu", "age": 20, "course": "BCA"}
print(student["name"])   # Output: Anu
```

**Common Dictionary Methods:**

1. keys() – returns all keys

```
student.keys()
```

2. values() – returns all values

```
student.values()
```

3. get() – returns value of a key safely

```
student.get("age")
```

**12. For Loop in Python**

A **for loop** is used to iterate over a sequence like list, tuple, string, or range.

**Syntax:**

```
for variable in sequence:
   statements
```

**Example Program (1 to 10):**

```
for i in range(1, 11):
   print(i)
```
**Output:**
1 2 3 4 5 6 7 8 9 10

**12. Describe the for loop in Python. Write a program to print numbers from 1 to 10.**

A for loop is used to iterate over a sequence such as list, tuple, string, or range.
Syntax: for variable in sequence: statements
Program: for i in range(1, 11): print(i)

13. Explain the "Arguments of an Exception" with an example.

Answer: When an exception occurs, it carries additional information known as arguments.
Example: try: x = int("abc") except Exception as e: print("Error:", e)
Explanation: The exception argument provides details about the error, helping in debugging.

14. Describe accessing characters in a string with examples.

Answer: Strings are sequences of characters and can be accessed using indexing and slicing.
Indexing: s = "Python" print(s[0]) # P print(s[-1]) # n
Slicing: print(s[1:4]) # yth

15. Distinguish between a Histogram and a Bar Chart based on their structure and data usage.

Answer: Histogram:

- Used for continuous data
- Bars are adjacent (no gaps)
- Shows frequency distribution

Bar Chart:

- Used for categorical data
- Bars have gaps between them
- Used for comparison

16. Write short notes on common string methods.

Answer: String methods are used to manipulate strings.
Examples: upper() – converts to uppercase lower() – converts to lowercase strip() – removes whitespace replace() – replaces characters

**17. Variables in Python**

A **variable** is a name used to store data.
**Example:**

```
x = 10
name = "Sensor"
```

**Rules:**

- Must start with a letter or underscore
- Cannot use keywords
- Case-sensitive

**Dynamic Typing:**

```
x = 10
x = "hello"
```

**Applications in Electronics:**

- Storing sensor values

```
temperature = 35.5
```

- Voltage/current calculation

```
voltage = 5
current = 2
```

power = voltage * current

- Signal processing values

**18. Loop Control Flow in Python**

**Loops** are used to repeat a block of code.

**For Loop:**

```
for i in range(5):
  print(i)
```

**While Loop:**

```
i = 1
while i <= 5:
  print(i)
  i += 1
```

**Loop Control Statements:**

1. **break** – exits loop

```
for i in range(5):
  if i == 3:
    break
```

2. **continue** – skips iteration

```
for i in range(5):
  if i == 3:
    continue
```

3. **pass** – does nothing

```
for i in range(5):
  pass
```

**19. Functions in Python**

A **function** is a reusable block of code.

**Definition:**

```
def add(a, b):
  return a + b
```

**Calling Function:**

```
result = add(2, 3)
print(result)
```

**Passing Arguments:**

- Positional arguments
- Keyword arguments

```
add(a=2, b=3)
```

**Return Statement:**

- Sends result back to caller

```
def square(x):
  return x*x
```

**Example:**

```
def greet(name):
  return "Hello " + name

print(greet("Anu"))
```

**20. Regular Expressions Workflow**

Regular expressions are used for **pattern matching**.

**Steps:**

1. Import module

import re

2. Define pattern

pattern = r"\d+"

3. Apply functions

- search() – finds first match

re.search(pattern, "abc123")

- match() – checks from start

re.match(pattern, "123abc")

- findall() – returns all matches

re.findall(pattern, "a1 b2 c3")

**Example:**

import re
text = "My number is 9876543210"
result = re.findall(r"\d+", text)
print(result)

**Output:**

['9876543210']

**Conclusion:**

- Regex is useful in validation, searching, and data extraction.